

Práctica 2: Algoritmo de Tomasulo: Etapas Issue y Writeback

Arquitecturas de computación avanzadas

Curso 24/25

1. Objetivos de la práctica

- Comprender el funcionamiento del algoritmo de gestión dinámica de instrucciones conocido como Algoritmo de Tomasulo.
- Implementar y evaluar dicho algoritmo.

2. Desarrollo de la práctica

Para el desarrollo de la práctica se utilizará el simulador *MARS-F* (versión con código fuente) disponible en el Campus Virtual de la asignatura. Esta versión del simulador implementa la ruta de datos del algoritmo de Tomasulo, aunque faltan por implementar las etapas *ISSUE* y *WB* del algoritmo.

La ruta de datos implementada difiere ligeramente de la presentada en las clases teóricas. En la Figura 1 podemos ver la ruta de datos del simulador. A continuación, se exponen las principales diferencias entre esta implementación y la vista en clase:

- Aunque *MARS* dispone del conjunto de instrucciones completo de *MIPS*, la ruta de datos implementada sólo utiliza operaciones aritméticas con enteros. Para simular el comportamiento de Tomasulo, se supondrá que todas las operaciones de multiplicación y división de enteros se comportan como operaciones de punto flotante.
- Las operaciones de multiplicación y división siempre modifican los registros *HI* y *LOW*. Por esa razón, estas instrucciones siempre marcan esos registros en el fichero de registros, además de un tercer registro en las instrucciones de multiplicación de tres operandos¹.
- Si una estación de reserva está esperando alguno de sus operandos, puede comenzar la ejecución de su instrucción (si el operador está libre) en el mismo ciclo que lee el operando desde el CDB (etapa Writeback de la instrucción que produce el operando).
- No se implementará ejecución especulativa, por lo que se usará un predictor de saltos ideal.
- Aunque no se implemente ejecución especulativa, se utilizará la estructura del Reorder Buffer para la implementación del algoritmo. Trabajar directamente con el Reorder Buffer nos ayudará a reducir el trabajo en la práctica 3.
- La implementación de la etapa *Commit* se limita a comprobar si la entrada en cabeza del Reorder Buffer ha completado su etapa *WB*, y en ese caso, actualiza el registro de salida y libera la entrada.

¹Las operaciones de división con tres operandos en realidad son pseudo-instrucciones compuestas de una división con dos operandos seguida de una instrucción *mflo*.

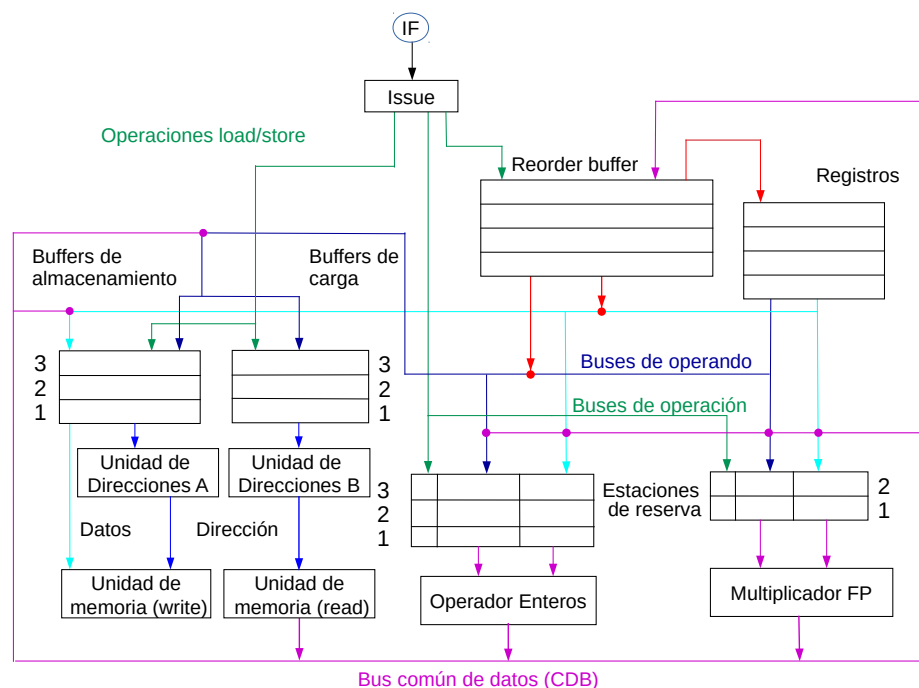


Figura 1: Ruta de datos del algoritmo de Tomasulo empleado en la práctica 2.

- La unidad de memoria difiere a la vista en clase. Por una parte, se disponen de dos unidades de cálculo de direcciones y de dos unidades de memoria: una para lecturas y otra para escrituras, permitiendo realizar una carga y un almacenamiento simultáneamente. Además, el cálculo de la dirección y la lectura/escritura se realizan en serie. Por tanto, en el caso de las operaciones de escritura, no se procederá al cálculo de la dirección hasta que se disponga tanto del registro de dirección como del registro a escribir.

El enunciado de la práctica se organiza como sigue: explicación de los principales paquetes y clases utilizadas en el simulador, la estructura de la unidad de gestión dinámica de instrucciones y el pseudocódigo del algoritmo de Tomasulo. Finalmente, se indican los ejercicios a realizar.

2.1. Entrega de la práctica

La fecha límite de entrega será el miércoles 9 de abril a las 23:55. Se debe entregar:

- Memoria respondiendo a todas las preguntas hechas en el guion de prácticas².
- El código fuente de los ficheros de las clases implementadas (`TomasuloIdeal_alumnos.java`, `Estacion.java` y las 4 clases heredadas de `Estacion.java`)

²Los formularios del PDF pueden no visualizarse con Adobe Acrobat Reader. Si eres usuario de Windows, mi recomendación es utilizar *Okular*, un visor de PDFs gratuito, de software libre y con más funcionalidades que la versión gratuita de Acrobat Reader. Puedes descargar *Okular* aquí: <https://okular.kde.org/es/download/>.

3. Estructura del simulador de Tomasulo

3.1. Paquetes y clases de java

La implementación del algoritmo de Tomasulo en *MARS-F* se compone de los siguientes paquetes y clases:

- Paquete `mars.pipeline`: Contiene diversas clases abstractas para la definición de los principales elementos del pipeline. Las clases que contiene son las siguientes:
 - `Pipeline`: Clase abstracta para la definición de rutas de datos de procesador.
 - `BranchPredictor`: Clase abstracta para la definición de predictores de saltos.
 - `Stage`: Enumeración con las posibles etapas de la ruta de datos.
 - `StageRegisters`: Registros inter-etapa. Se utilizan para que *MARS* se comunice con la ruta de datos de Tomasulo y para comunicar las etapas *IF* e *Issue*. Contienen la dirección y la instrucción a ejecutar, así como diversa información para la posterior ejecución de las instrucciones y visualización de los datos.
 - `Decode`: Clase con múltiples métodos estáticos para decodificación de instrucciones. Por ejemplo:
 - `Decode.hasRs()` devuelve true si la instrucción tiene un primer operando, false en caso contrario.
 - `Decode.getRs()` devuelve el identificador del primer registro operando si la instrucción lo tiene, o cero en caso contrario.
 - `Decode.getDestination()` devuelve el identificador del registro destino si la instrucción lo tiene, o cero en caso contrario. La función devuelve siempre el registro correcto, sea *Rt* (instrucciones tipo I) o *Rd* (instrucción tipo R).
 - `Decode.getFormat()` devuelve el tipo de formato de la instrucción usando la clase *BasicInstructionFormat*, cuya salida puede ser:
 - ◊ `R_FORMAT`: instrucciones tipo R.
 - ◊ `I_FORMAT`: instrucciones tipo I con registro destino, es decir, loads, stores y aritmético-lógicas, las cuales poseen un registro operando (*Rs*), un inmediato y un registro destino (*Rt*).
 - ◊ `I_BRANCH_FORMAT`: instrucciones tipo I de bifurcación, las cuales poseen dos registros operandos (*Rs* y *Rt*) y no poseen registro de destino.
 - ◊ `J_FORMAT`: instrucciones tipo J.

Se recomienda encarecidamente leer la documentación de esta clase, ya que tiene todos los métodos necesarios para obtener la información de decodificación de una instrucción.
- `pipe_config`: Configura el pipeline a través del un fichero *xml*.
- Paquete `mars.pipeline.pipes`: Contiene la implementación de las rutas de datos sin planificación dinámica de instrucciones que fueron utilizadas en la práctica 1 (procesador *MIPS* segmentado de 5 etapas y con unidades multiciclo).
- Paquete `mars.pipeline.predictors`: Contiene la implementación de diversos predictores de saltos:
 - `IdealPredictor`: Predictor ideal que siempre acierta.
 - `StopPredictor`: Predictor que soluciona los riesgos de control deteniendo la ejecución (en realidad no predice nada).
 - `NotTakenPredictor`: Predice siempre no tomado.
 - `TakenPredictor`: Predice siempre tomado.

- `DynamicPredictor`: Clase abstracta para implementar predictores dinámicos.
 - `BitPredictor`: Implementa una “*Branch Prediction Table*” con predictores dinámicos de 1, 2 o 3 bits.
 - `BTB`: Implementa un predictor dinámico conocido como “*Branch Target Buffer*” (*BTB*). El funcionamiento de la *BTB* se repasará más adelante de cara a su utilización en el práctica 3.
- Paquete `mars.pipeline.tomasulo`: Contienen las clases que implementan la ruta de datos con el Algoritmo de Tomasulo.
- Clases necesarias para la realización de esta práctica:
 - `CDB`: Implementa el buffer común de datos o CDB. Ofrece métodos estáticos para acceder a la única instancia del CDB.
 - `registro`: Implementa el banco de registros. Ofrece métodos estáticos para acceder a la única instancia del banco de registros.
 - `Estacion`: Clase abstracta para definir estaciones de reserva. Contiene dos métodos abstractos a implementar en cada subclase: *Execute()* y *Ocupar()*. *Execute()* ejecuta un ciclo de la instrucción asociada a la estación de reserva y ya está implementado. *Ocupar()* guarda una instrucción en una estación de reserva libre. Las implementaciones de los métodos *Ocupar()* en las subclases de `Estacion` serán realizadas durante esta práctica. También habrá que implementar los métodos *checkQj()* y *checkQk*, que servirán para comprobar la disponibilidad de los operadores de cada instrucción.
 - `ReorderBuffer`: Clase que implementa la estructura de datos y métodos para manejar el buffer de reordenamiento o “*Reorder Buffer*” (*rob*). Ofrece un método estático para acceder a la única instancia del *rob*.
 - `RobEntry`: Clase que implementa la estructura de datos y métodos para manejar una entrada del *rob*.
 - `Tomasulo`: Clase abstracta que define los métodos para la ejecución de la ruta de datos de Tomasulo e implementa las estructuras de datos necesarias para su funcionamiento.
 - `Tomasulo_conf`: Contiene todos los parámetros de configuración de la ruta de datos Tomasulo, como la latencia de los operadores, número de estaciones de reserva de cada tipo, etc.
 - `TomasuloIdeal_alumnos`: Subclase que implementa el algoritmo de Tomasulo que se utilizará en la práctica 2. Todos los métodos están implementados excepto los métodos *Issue()* y *Writeback()*, que serán implementados durante esta práctica.
 - Otras clases necesarias para la realización de esta práctica pero con las que no será necesario interactuar durante el desarrollo de la misma:
 - `Tomasulo_alumnos`: Subclase de `Tomasulo` que declara las estaciones de servicio utilizando las subclases a implementar por los alumnos. Su única misión es establecer una jerarquía de clases para distinguir entre las clases a implementar y las ya implementadas (no disponibles en esta versión del simulador pero si en la versión completa sin código fuente).
 - `InstructionSetTomasulo`: Implementa el set de instrucciones *MIPS* que soporta nuestra ruta de datos Tomasulo.
 - `Operador`: Implementa un operador genérico. Como en realidad la ejecución de las instrucciones la realiza la clase `InstructionSetTomasulo`, basta con un único operador para todos los tipos de operación.
 - Clases no necesarias que serán utilizadas en la práctica 3:

- `TomasuloEspeculacion_alumnos`: Subclase que implementa el algoritmo de Tomasulo con especulación que se utilizará en la práctica 3. Actualmente la clase está vacía y su uso conllevará el lanzamiento de una excepción.
- Paquete `mars.pipeline.tomasulo.DiagramaMulticiclo`: Incluye clases para la gestión del diagrama de ejecución multiciclo.
 - `DiagramaMulticiclo`: Clase que implementa el diagrama de ejecución multiciclo.
 - `InstructionInfo`: Clase que guarda diversa información de las instrucciones que se utilizará para la posterior visualización del diagrama multiciclo.
 - `HtmlUtils`: Clase con múltiples métodos estáticos para la generación de estructuras html utilizadas para generar la información de profiling de la ruta de datos.
- Paquete `mars.pipeline.tomasulo.estaciones_alumnos`: Contiene las subclases de `Estacion` que implementan las diversas estaciones de reservas utilizadas:
 - `EstacionINT_alumnos`: clase para estaciones de reserva de operaciones de enteros.
 - `EstacionPF_alumnos`: clase para estaciones de reserva de operaciones de punto flotante. Además de los métodos heredados, posee nuevos métodos para gestionar los registros HI y LOW.
 - `EstacionSW_alumnos`: clase para estaciones de reserva de operaciones de instrucciones de almacenamiento en memoria (*sw*).
 - `EstacionLW_alumnos`: clase para estaciones de reserva de operaciones de instrucciones de carga desde memoria (*lw*).

Todas las clases necesarias para la realización de la práctica incluyen la documentación en formato JavaDOC. En el Campus Virtual de la asignatura se puede descargar toda la documentación generada por el proyecto. También puede generarse desde el IDE de programación utilizado. Notar que la documentación relevante para la realización de la práctica será únicamente la incluida en el paquete `mars.pipeline`.

3.2. Estructura de la unidad de gestión dinámica de instrucciones (clase Tomasulo)

La unidad de gestión dinámica está compuesta por los siguientes elementos:

- **Estaciones de Reserva de operaciones de enteros.** Contiene las estaciones de reserva del operador de enteros. Está representada por la variable `est_Enteros`. El número de estaciones de reserva viene indicado por la constante `NUM_EST_ENTEROS`³ (por defecto, 4).
- **Operador Enteros.** Se encarga de realizar las operaciones de enteros. Está representado por la variable `opEnteros`. La latencia en ciclos del operador viene indicada por la constante `LAT_ENTEROS` (por defecto, 1).
- **Estaciones de Reserva de operaciones de punto flotante.** Contiene las estaciones de reserva del operador de punto flotante. Está representada por la variable `est_PF` y la constante `NUM_EST_PF` indica el número de estaciones de reserva (por defecto, 2).
- **Operador de coma flotante.** Se encarga de realizar las operaciones de coma flotante. Está representado por la variable `opPF`. La latencia en ciclos del operador viene indicada por la constante `LAT_PF` (por defecto, 6). El operador no está segmentado.
- **Buffers de escritura o almacenamiento.** Contiene las estaciones de reserva para las operaciones de escritura en memoria. Está representada por la variable `est_Store`. El número de buffers viene indicado por la constante `NUM_EST_STORE` (por defecto, 3).

³Las constantes indicadas a lo largo de esta sección se encuentran en la clase `Tomasulo_conf`.

- **Unidad de escritura de memoria.** Se encarga de realizar las operaciones de almacenamiento en memoria. Está representado por la variable `opStore`. La latencia en ciclos de la unidad viene indicada por la constante `LAT_STORE` (por defecto, 3 ciclos: 1 para el cálculo de la dirección y 2 para la escritura en memoria). El operador no está segmentado.
- **Buffers de lectura o carga.** Contiene las estaciones de reserva para las operaciones de lectura en memoria. Está representada por la variable `est_Load`. El número de buffers viene indicado por la constante `NUM_EST_LOAD` (por defecto, 3).
- **Operador de lectura de memoria.** Se encarga de realizar las operaciones de lectura de memoria. Está representado por la variable `opLoad`. La latencia en ciclos de la unidad viene indicada por la constante `LAT_LOAD` (por defecto, 3 ciclos: 1 para el cálculo de la dirección y 2 para la escritura en memoria). El operador no está segmentado.
- Además, existe una variable llamada `est_Todas` que contiene todas las estaciones de reserva. Puede ser de utilidad si se desea recorrer todas las estaciones para realizar una tarea común a todas, independientemente de la subclase.
- **Buffer de Reordenamiento o Reorder Buffer.** Se encarga de la gestión del *rob* y está representado por la variable `rob`. Sin embargo, se puede acceder al *rob* desde cualquier clase a través del método estático `ReorderBuffer.getRoBInstance`⁴. El número de entradas del Reorder Buffer viene indicado por la constante `NUM_ENTRADAS_ROB` (por defecto, 20).

Además, aunque no son variables de la clase *Tomasulo*, caben destacar:

- **Bus común de datos o CDB:** Se encarga de las transferencias entre los diversos componentes del sistema. Se puede acceder al CDB desde cualquier clase a través del método estático `CDB.read()`.
- **Banco de registros.** Se puede acceder al banco de registros desde cualquier clase a través del método estático `registro.getRegs()` o a un solo registro a través del método estático `registro.getReg()`. Además, podemos acceder a las constantes `NUM_REGISTROS` (número de registros), `HI` (identificador registro `$hi`) y `LOW` (identificador registro `$low`).

Por último, señalar que la clase *Tomasulo.conf* contiene la constante `ES_NULO`, que nos servirá para marcar las etiquetas de estaciones, operadores y registros cuando no estén en uso o estén esperando una actualización.

⁴En realidad, la variable `rob` se obtiene a través de método `ReorderBuffer.getRoBInstance` durante la construcción del objeto de la clase *Tomasulo*.

3.3. Pseudocódigo del algoritmo de Tomasulo

A continuación se muestra el pseudocódigo del algoritmo de Tomasulo, descompuesto en sus fases principales.

3.3.1. Etapa Issue

Etapa de emisión de la instrucción. Se implementará en el fichero fuente `TomasuloIdeal_alumnos.java`

Input: Instrucción I

Output: true si se ha completado la emisión; false en caso contrario

```
//buscamos una entrada libre en el reorder buffer y una estación de reserva libre
Si {r: entrada en el Reorder Buffer} libre y
    {s: estación de reserva o buffer del tipo I.tipo} libre; entonces
    //añadimos la instrucción a la entrada del ROB
    r.añadir(I)
    //añadimos la instrucción a la estación seleccionada
    s.Ocupar(I)
    //Indicamos a la entrada del Rob la estación donde está la instrucción
    r.estacion := s
    //emisión completada
    return true
Sino
    return false
```

3.3.2. Función checkQj(int Rs)

Esta función comprueba la disponibilidad del operando *Rs*, actualizando el valor del operando en la estación de reserva si este está disponible, o marcando la entrada del rob que producirá el resultado en caso contrario.

Input: Número de registro Rs, Estación de reserva s

```
Si Rs != 0; entonces
    //comprobamos primero el registro
    Si registro[Rs].Qi == ES_NULO; entonces
        s.Vj := registro[Rs].valor
        s.Qj := ES_NULO
    Sino
        //comprobamos si la entrada del Rob que actualiza Rs
        //ha completado la etapa writeback
        Si rob[registro[Rs].Qi].WB_done; entonces
            s.Vj := rob[registro[Rs].Qi].valor
            s.Qj := ES_NULO
        Sino
            //debemos marcar la estación de reserva con la entrada del rob
            s.Qj := registro[Rs].Qi
Sino
    //no hay primer operador o es el registro $0
    s.Vj := 0
    s.Qj := ES_NULO
```

3.3.3. Función checkQk(int Rt)

Esta función comprueba la disponibilidad del operando *Rt*, actualizando el valor del operando en la estación de reserva si este está disponible, o marcando la entrada del rob que producirá el resultado en caso contrario.

Input: Número de registro *Rt*, Estación de reserva *s*

```

Si Rt != 0; entonces
    //comprobamos primero el registro
    Si registro[Rt].Qi == ES_NULO; entonces
        s.Vk := registro[Rt].valor
        s.Qk := ES_NULO
    Sino
        //comprobamos si la entrada del Rob que actualiza Rt
        //ha completado la etapa writeback
        Si rob[registro[Rt].Qi].WB_done; entonces
            s.Vk := rob[registro[Rt].Qi].valor
            s.Qk := ES_NULO
        Sino
            //debemos marcar la estación de reserva con la entrada del rob
            s.Qk := registro[Rt].Qi
Sino
    //no hay primer operador o es el registro $0
    s.Vk := 0
    s.Qk := ES_NULO

```

3.3.4. Función Ocupar(instrucción I)

Esta función comprueba los operandos de la instrucción *I* y actualiza el fichero de registros si la instrucción escribe un registro. Hay que tener en cuenta que los registros origen y destino dependerán del tipo de instrucción.

Este pseudocódigo modela la ocupación de una estación genérica que sirve para todos los tipos de instrucción (carga, almacenamiento, entero y punto flotante). Sin embargo, cada subclase de estación implementa su propia versión de esta función. Por lo tanto, habrá comprobaciones y partes del pseudocódigo que no serán necesarias incluir en la implementación final. Por ejemplo, no es necesario comprobar el tipo de instrucción: si estamos en una estación de enteros, la instrucción únicamente puede ser de tipo entero. También podemos omitir el comprobar si el registro *Rt* es un operando fuente en una estación de carga (*lw*), ya que esto nunca va a ocurrir, aunque si es necesario obtener el desplazamiento para calcular la dirección de memoria.

Input: Instrucción *I*, Estación de reserva *s*,
 identificador de entrada del rob *rob_id*, ciclo de ejecución *ciclo*
 Output: true si se ha ocupado la estación; false en caso contrario

```

Si s.ocupada; entonces
    return false

/*Actualizamos información básica de la estación*/
s.ocupada := true
s.rob := rob_id
s.marca_tiempo := ciclo
s.instruccion := I
s.estado := 0

```



```

/*ahora se deben procesar los operandos.
Operando 1: siempre se procesa*/
op1 := getOperando1(I)
checkQj(op1)

/*Operando 2: este operando está presente las operaciones de punto flotante;
en los stores y las de enteros si son de formato R o formato salto condicional.*/
op2 := getOperando2(I)
Si I.tipo == pf o I.tipo == store o
    (I.tipo == entero y I.formato == FORMAT_R) o
    (I.tipo == entero y I.formato == FORMAT_I_BRANCH); entonces
        checkQk(op2)
Sino
    /*Es posible que la instrucción sea de enteros tipo I.
    En ese caso; Vk sería el inmediato con extensión de signo*/
    Si I.tipo == enteros y I.formato = FORMAT_I; entonces
        s.Vk := ExtensionSigno(I)
    Sino
        s.Vk := 0
    s.Qk := ES_NULO

/*En casos de cargas o almacenamientos, se debe guardar el desplazamiento*/
Si I.tipo == carga o I.tipo == almacenamiento; entonces
    s.Inm := ExtensionSigno(I)

/*Por último, debemos marcar el fichero de registros
si la instrucción escribe en un registro (enteros; pf y almacenamiento)*/
dest := getDestination(I)
Si I.tipo != store y dest != 0; entonces
    registro[dest].Qi := rob_id

/*ATENCIÓN: CASO ESPECIAL. Si la operación es de punto flotante,
se deben marcar explícitamente los registros HI y LOW*/
Si I.tipo == pf; entoces
    registro[HI].Qi := rob_id
    registro[LOW].Qi := rob_id

```

3.3.5. Etapa EX (función Execute())

Esta función realiza la computación de las instrucciones. Ya está implementada en todas las estaciones y no es necesario modificarla.

```

Para {cada tipo de operador op}; hacer
    Estacion s := op.getEstacion();
    Si s != null; entonces
        op.execute(s);
        Si s.finalizada == true; entonces
            op.libera();

```

3.3.6. Etapa Writeback

Etapa writeback. Se implementará en el fichero fuente `TomasuloIdeal_alumnos.java`

```
//Primer paso.  Buscar la estacion de reserva lista para volcar datos
//y con la marca de tiempo más antigua
Estacion s := getEstaciónListaMasAntigua()

//leemos el CDB
cdb = CDB.read();

//Si no hemos encontrado una estación para escribir,
//marcamos el CDB con la marca nula y terminamos
Si s == null; entonces
    cdb.marca = ES_NULO
    return

//Si no, actualizamos el CDB
cdb.marca := s.rob_id
cdb.valor := s.resultado

//revisamos todas las estaciones de reserva por si están esperando ese valor
Para (todas las Estaciones de reserva check) hacer
    Si check.ocupada; entonces
        Si check.Qj == cdb.marca; entonces
            check.Vj := cdb.valor
            check.Qj := ES_NULO
        Si check.Qk == cdb.marca; entonces
            check.Vk := cdb.valor
            check.Qk := ES_NULO

//actualizamos el contenido del Reorder Buffer
rob[cdb.marca].valor := cdb.valor
rob[cdb.marca].WB_done := true;
//CASO ESPECIAL: si la operación es de punto flotante,
//hay que escribir explícitamente HI y LOW
Si s es una instancia de EstacionPF; entonces
    rob[cdb.marca].HI := s.HI
    rob[cdb.marca].LOW := s.LOW

//Liberamos la estación
s.ocupada := false;
```

3.3.7. Etapa Commit

Como se ha comentado anteriormente, la etapa Commit se limita a comprobar si la instrucción en cabeza del rob ha ejecutado la etapa Writeback para actualizar el registro de salida y liberar su entrada en el rob.

4. Ejercicios a realizar

- 1 Implementación del algoritmo de Tomasulo. Tras familiarizarse con las estructuras de datos y la estructura del simulador, se debe implementar las fases ISSUE y WB del algoritmo de Tomasulo para las operaciones aritméticas. Para ello, se implementarán las funciones Issue y WB en el fichero `TomasuloIdeal_alumnos.java` y el método `Ocupar()` en las cuatro subclases de Estacion (enteros, punto flotante, carga y almacenamiento).

Para obtener la información necesaria para ejecutar una instrucción determinada, como los operandos fuente, registro destino, inmediato con extensión de signo, tipo de instrucción, etc., se recomienda encarecidamente utilizar la clase `Decode`. Esta clase contiene funciones para obtener toda esta información. La documentación de los métodos de la clase `Decode` se puede consultar en la documentación `JavaDoc` del proyecto.

- 2 Comprobar el funcionamiento del algoritmo de Tomasulo. Una vez implementado y compilado el algoritmo de Tomasulo, se comprobará su funcionamiento mediante el siguiente ejemplo:

```
.data

x: 10
y: 5
z: 25
store: .space 8

.text

lw $t0, x           #cargamos los tres valores
lw $t1, y           #pseudo-instrucciones
lw $t2, z           #se traducen por lui+lw
add $t3, $t0, $t1    # (x+y)
mul $t4, $t2, $t3    # (x+y)*z
srl $t4, $t4, 2      # (x+y)*z/4
la $t5, store        #cargamos dir. de guardado
sw $t3, 0($t5)       #guardamos ambos resultados
sw $t4, 4($t5)

li $v0, 10           #terminamos el programa
syscall
```

El fichero `TomasuloIdeal.xml` de los ejemplos proporcionados en el Campus Virtual contiene todos los parámetros configurables de este pipeline. Para utilizar el pipeline Tomasulo con la configuración por defecto, basta con crear un fichero `xml` con el siguiente contenido:

```
<?xml version="1.0" encoding="utf-8"?>
<pipeline tipo="tomasulo" especulacion="no" limite="100" ventana="25">
</pipeline>
```

Al ejecutar el simulador se generarán diversos ficheros en formato `html` por cada ciclo con la información sobre el estado de la máquina y el diagrama de ejecución multiciclo, además de un resumen con las instrucciones ejecutadas y su tipo. Para visualizar los ficheros se puede utilizar cualquier navegador.

Para comprobar el correcto funcionamiento del algoritmo, se puede utilizar la versión del simulador con el algoritmo de Tomasulo implementado suministrada en el Campus Virtual de la asignatura.

- 3 Comprobar el funcionamiento de una iteración de un bucle SDOT ($k = \vec{X} \cdot \vec{Y}$). Para ello usaremos el fichero `sdot.asm`. En cada iteración del bucle SDOT, se leen dos vectores de dos componentes de dos arrays, y se calcula el producto escalar de los dos vectores. El resultado del producto escalar se guarda en un tercer array.

Antes de continuar, se recomienda comprobar el correcto funcionamiento del simulador con la configuración inicial de los operadores. Una vez comprobado, modificaremos el procesador para tener 4 estaciones de reserva de punto flotante y 4 buffers de carga, manteniendo el número de buffers de almacenamiento y de estaciones de reserva de enteros en su valor por defecto. Para ello, editaremos el fichero `xml` del pipeline añadiendo los parámetros necesarios:

```
<?xml version="1.0" encoding="utf-8"?>
<pipeline tipo="tomasulo" especulacion="no" limite="100" ventana="25">
    <estaciones_lw>4</estaciones_lw>
    <estaciones_pf>4</estaciones_pf>
</pipeline>
```

Indica cuál es el tiempo de ejecución total (en ciclos), el número de instrucciones ejecutadas y CPI del programa `sdot.asm`.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Seguidamente, vamos a aumentar el tamaño de los arrays del programa `sdot.asm` a 30 vectores de dos componentes, usando para ello el fichero `sdot30.asm`. Indica el tiempo de ejecución total (en ciclos), el número de instrucciones ejecutadas y CPI. ¿A qué se debe la reducción del CPI respecto a la ejecución de una sola iteración?

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Observaciones:

- 4 Comprobar el funcionamiento de `sdot30.asm` con 4 buffers de carga y manteniendo el resto de la configuración del procesador en sus valores por defecto. Indica el tiempo de ejecución total (en ciclos), el número de instrucciones ejecutadas y CPI. ¿A qué se debe la diferencia de rendimiento respecto a la ejecución del programa `sdot30.asm` en el ejercicio 3?

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Observaciones:

- 5 Utilizando la configuración del procesador del ejercicio 3, comprueba el funcionamiento del programa `sdot30-v2.asm`. El programa realiza el mismo trabajo que `sdot30.asm`, pero realiza las operaciones en un orden diferente. Indica el tiempo de ejecución total (en ciclos), el número de instrucciones ejecutadas y CPI. ¿A qué se debe la diferencia de rendimiento entre ambos programas? ¿Qué conclusiones extraes de esto?

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Observaciones: